



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Ricerca in ambienti complessi

Ingegneria Informatica

Docente

Domenico Fabio Savo

Intelligenza Artificiale

Ricerca non informata vs ricerca informata

Nei problemi di ricerca visti (fino ad ora) l'obiettivo è trovare **cammini attraverso lo spazio degli stati** che portano dallo stato iniziale a quello obiettivo.

Ad esempio il cammino da Arad a Bucarest.

In alcuni casi siamo **interessati soltanto allo stato obiettivo**, non al cammino percorso per raggiungerlo.

Alcuni esempi sono la progettazione di circuiti integrati, il layout di impianti industriali, la programmazione automatica, l'ottimizzazione di reti di telecomunicazioni.



Algoritmi costruttivi e algoritmi di ricerca locale

- Quelli visti fino ad ora, prendono il nome di **algoritmi costruttivi** poiché generano una soluzione ex-novo aggiungendo ad una situazione di partenza (vuota o iniziale) delle componenti in un particolare ordine fino a “costruire” la soluzione completa (la sequenza di operatori che dallo stato iniziale ci portano allo stato obiettivo)
- Gli **algoritmi di ricerca locale**, invece, partono da una soluzione iniziale e iterativamente cercano di sostituire la soluzione corrente con una **migliore** in un intorno della soluzione corrente. In questi casi non interessa il **percorso** per raggiungere l’obiettivo.

Diversi problemi (ad esempio, il problema delle 8 regine) hanno la proprietà che la descrizione stessa dello stato contiene tutte le informazioni necessarie per la soluzione (il cammino è quindi irrilevante).

In questo caso gli algoritmi con **miglioramenti iterativi** spesso forniscono l’approccio più pratico.

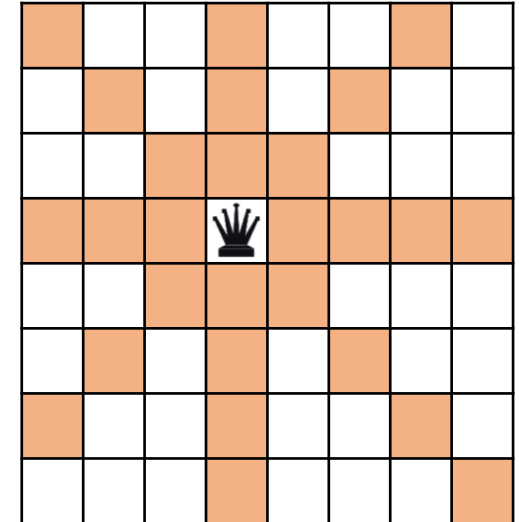


Il problema delle 8 regine

È data una scacchiera (8x8): il problema consiste nel posizionarvi otto regine in modo da evitare un attacco reciproco.

Le mosse possibili per la regina prevedono tutte le posizioni sulla stessa riga, colonna e diagonale a partire dalla casella corrente.

In questo caso gli algoritmi con miglioramenti iterativi spesso forniscono l'approccio più pratico.



Algoritmi di ricerca locale

Gli algoritmi di **ricerca locale** operano cercando a partire da uno stato iniziale e procedendo verso gli stati adiacenti.

- Non tengono traccia dei cammini
- Non tengono traccia degli stati già raggiunti

Quindi non sono sistematici: potrebbero anche non esplorare mai una porzione dello spazio degli stati dove risiede una soluzione.

Hanno però due **vantaggi**:

- (1) usano pochissima memoria
- (2) spesso riescono a trovare soluzioni **ragionevoli** in spazi degli stati grandi o anche infiniti per cui usare algoritmi sistematici non sarebbe praticabile.

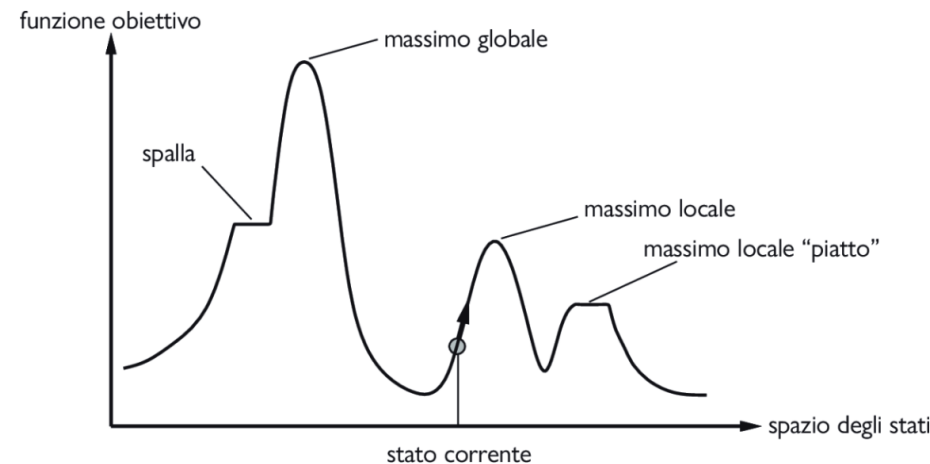


Panorama dello spazio degli stati

Possiamo considerare gli stati di un problema come disposti in un panorama dello spazio degli stati.

Ogni punto (stato) nel panorama ha una “altezza” definita dal valore della **funzione obiettivo**

- Se l’altezza corrisponde a una funzione obiettivo, lo scopo è quello di trovare il picco più alto (**un massimo globale**) e chiamiamo questo processo **hill climbing**.
- Se l’altezza corrisponde al costo, lo scopo è quello di trovare l’avvallamento più profondo (**un minimo globale**) e parliamo in questo caso di **discesa del gradiente**.



Ricerca Locale (1\2)

La **ricerca locale** è basata sull'esplorazione iterativa di **soluzioni vicine**, che possono migliorare la soluzione corrente mediante modifiche locali.

- **Struttura dei “vicini” (neighborhood):** Una struttura dei “vicini” è una funzione **N** che assegna a ogni soluzione **s** dell'insieme di soluzioni **S** un insieme di soluzioni **N(s)** sottoinsieme di **S**.
- La scelta della funzione **N** è fondamentale per l'efficienza dell'approccio e definisce l'insieme delle soluzioni che possono essere raggiunte da **s** in un singolo passo di ricerca dell'algoritmo. Tipicamente è definita implicitamente attraverso le possibili mosse.
- La soluzione trovata da un algoritmo di ricerca locale **non è detto sia ottima globalmente**, ma può essere ottima rispetto ai cambiamenti locali (**ottimo locale**)



Ricerca Locale (2\2)

Sia f una funzione **euristica** di **costo di stato** (non di cammino).

Massimo (minimo) locale: data una funzione di valutazione f , un **massimo (minimo)** locale è una soluzione s tale che per qualunque s' in $N(s)$ si ha $f(s) \geq f(s')$. ($f(s) \leq f(s')$)

Quando risolviamo un problema di massimizzazione (minimizzazione) cerchiamo un massimo (minimo) globale s_{opt} cioè tale che per qualunque s abbiamo che $f(s_{opt}) > f(s)$

Più è largo il neighborhood ($N(s)$) più è probabile che un massimo locale sia anche un massimo globale

- L'algoritmo base a cui ci si riferisce è detto “**iterative improvement**” (miglioramento iterativo):
L'algoritmo parte con una soluzione di tentativo (generata in modo casuale o mediante un procedimento costruttivo) e cerca di migliorare la soluzione corrente muovendosi verso i vicini. Se fra i vicini c'è una soluzione migliore, tale soluzione va a rimpiazzare la corrente. Altrimenti termina in un massimo (minimo) locale.



Algoritmo Hill Climbing

Un algoritmo di ricerca locale molto noto è l'**hill climbing**.

- L'algoritmo tiene traccia di un solo stato corrente e a ogni iterazione passa allo stato vicino con valore più alto (basso) senza guardare oltre gli **stati immediatamente vicini** a quello corrente.
- Hill climbing si muove verso i valori più alti (o bassi) che corrispondono a massimi (o minimi) locali.
- **Problema:** l'algoritmo può incappare in un massimo (minimo) locale di scarsa qualità: ha il limite di essere un metodo locale che non valuta tutta la situazione, ma solo gli stati abbastanza vicini a quello corrente.

L'algoritmo non gestisce un albero di ricerca, ma solo lo stato e la sua valutazione.



Algoritmo hill-climbing

Dato uno stato *corrente*, l'algoritmo esamina i suoi stati *vicini*. Se ne trova uno con una **funzione di valutazione** più alta (o bassa se sto cercando un minimo) allora rende tale stato il nuovo stato *corrente*. Continua così finché non si trova in uno stato *corrente* che non ha vicini migliori. Notare la *mancaanza di una frontiera*

```
function Hill-Climbing(problema) returns uno stato che è un massimo locale
    corrente ← problema.StatoIniziale
    while true do
        vicino ← lo stato successore di corrente con valore più alto
        if Valore(vicino) ≤ Valore(corrente) then return corrente
        corrente ← vicino
```



Massimi locali, creste e plateau (1\2)

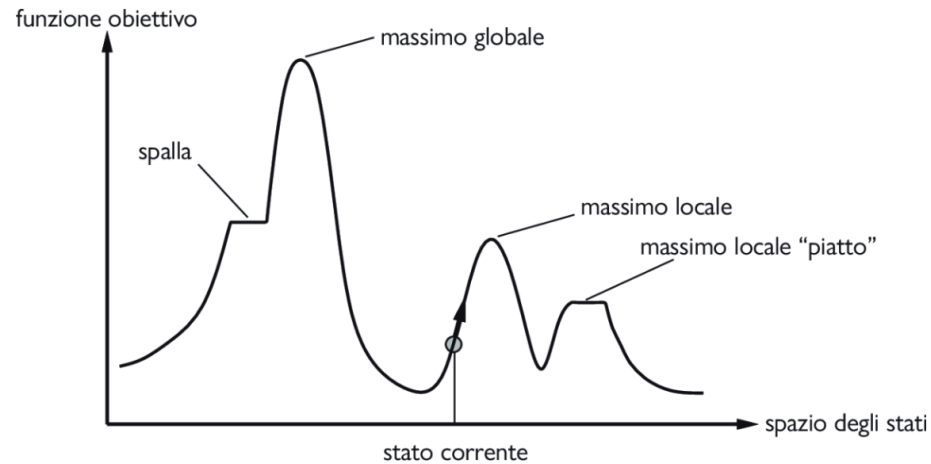
L'algoritmo **hill climbing** viene talvolta chiamato **ricerca locale greedy**, perché sceglie uno stato vicino “buono” senza pensare a come andrà avanti.

Spesso l'hill climbing rimane bloccato per le seguenti ragioni:

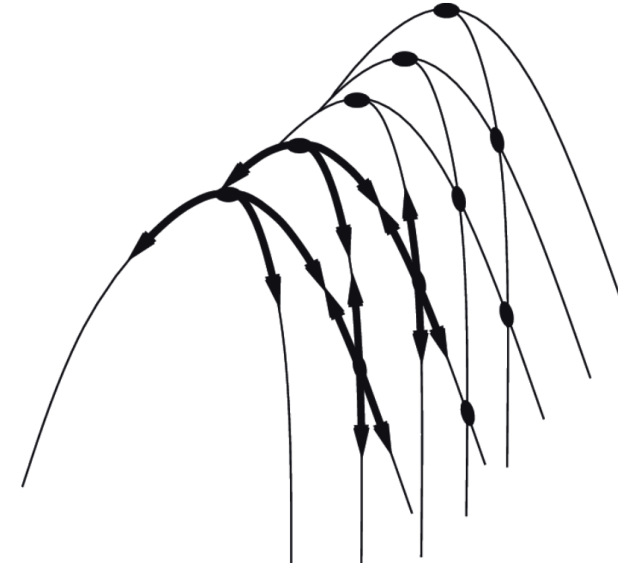
- **massimi locali:** un massimo locale è un picco più alto degli stati vicini, ma inferiore al massimo globale. Gli algoritmi hill climbing che raggiungono la vicinanza di un massimo locale saranno attirati verso il picco, ma rimarranno bloccati lì senza poter andare altrove.
- **creste:** Le creste danno origine a una sequenza di massimi locali molto difficili da esplorare da parte degli algoritmi greedy;
- **plateau:** un plateau è un'area piatta del panorama dello spazio degli stati. Può essere un massimo locale piatto, da cui non è possibile fare ulteriori progressi, oppure una spalla (shoulder), da cui si potrà salire ulteriormente.



Massimi locali, creste e plateau (2\2)



Esempio di massimo locale e plateau



Esempio di creste

Una possibile (semplice) soluzione per questi casi è ripartire con una nuova ricerca da una **soluzione di partenza casuale** o **generata in modo costruttivo**. Si salva poi la soluzione migliore dopo una serie di tentativi (bound fissato sul numero di tentativi o sul tempo di calcolo).

Hill climbing: il problema delle 8 regine

Problema: Inserire 8 regine in una scacchiera 8x8 in modo che non si attacchino

Costo **f**: numero di coppie di regine che si attaccano a vicenda. I numeri sono i valori negli stati successivi.

Per 8 regine, mediamente l'algoritmo si blocca l'86% delle volte, risolvendo solo il 14% delle istanze del problema.

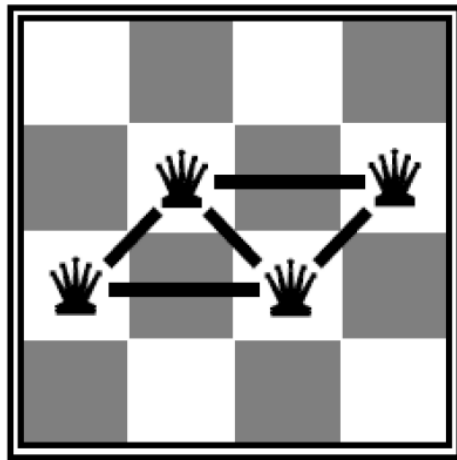
Richiede in media solo 4 passi per arrivare a una soluzione (solo 3 passi se si blocca).

Lo spazio di ricerca ha 8^8 nodi (circa 17 milioni di nodi) impraticabile la ricerca di qualsiasi tipo

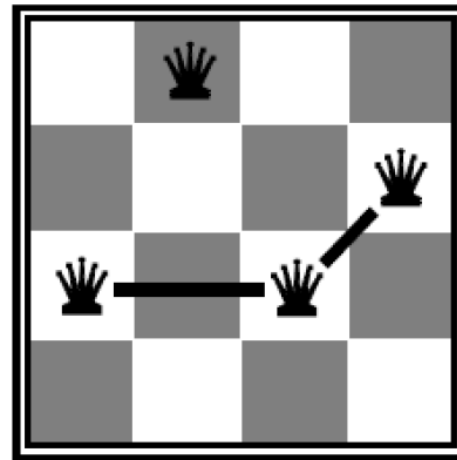
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

Hill climbing: esempio 4 regine

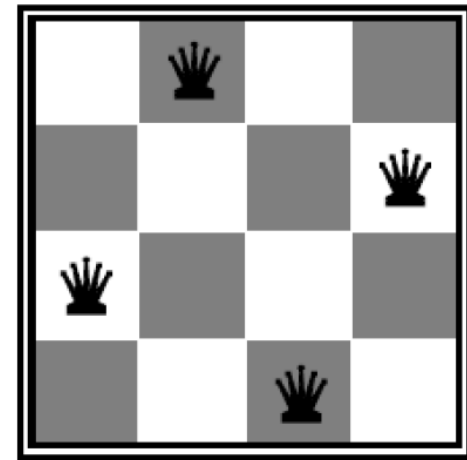
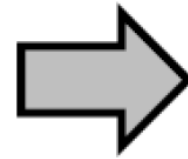
Problema: Inserire 4 regine in una scacchiera 4x4 in modo che non si attacchino



$f = 5$



$f = 3$



$f = 0$

In questo caso l'algoritmo è in grado di individuare la soluzione in 3 mosse.

Algoritmo Simulated Annealing

- L' algoritmo **Simulated annealing** (tempra simulata) combina hill-climbing con una scelta stocastica (ma non del tutto casuale, perché sarebbe poco efficiente)
- Il nome deriva dall'analogia con il processo di tempra dei metalli in metallurgia in cui un metallo viene portato ad alta **temperatura** per poi farlo raffreddare lentamente.
- La **temperatura T** rappresenta il grado con cui si è disposti ad accettare una scelta casuale **anche se non migliorativa**.



Algoritmo Simulated Annealing: Idea

Idea: ad ogni passo si sceglie un successore a caso:

- se migliora lo stato corrente viene espanso
- se no, ovvero $\Delta E = f(n') - f(n) < 0$, il nodo viene scelto con probabilità
$$p = e^{\Delta E / T} \text{ con } 0 \leq p \leq 1$$

[Si genera un numero casuale tra 0 e 1: se questo è minore di p il successore viene scelto, altrimenti no]

- p è inversamente proporzionale al peggioramento (ΔE negativo)
- T decresce col progredire dell'algoritmo (quindi anche p) secondo un piano definito



Algoritmo Simulated Annealing: comportamento

La probabilità di una mossa in discesa diminuisce col tempo e l'algoritmo si comporta sempre di più come Hill Climbing.

Se T viene decrementato abbastanza lentamente con probabilità tendente ad 1 si raggiunge la soluzione ottimale.

Analogia col processo di tempra dei metalli:

- T corrisponde alla temperatura
- ΔE alla variazione di energia



Algoritmo Simuated Annealing: Algoritmo

```
function Simulated-Annealing(problema, velocità_raffreddamento) returns uno stato soluzione
  corrente  $\leftarrow$  problema.Statoiniziale
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  velocità_raffreddamento[t]
    if T = 0 then return corrente
    successivo  $\leftarrow$  un successore di corrente scelto a caso
     $\Delta E \leftarrow$  Valore(corrente) – Valore(successivo)
    if  $\Delta E > 0$  then corrente  $\leftarrow$  successivo
    else corrente  $\leftarrow$  successivo solo con probabilità  $e^{\Delta E/T}$ 
```

L'input *velocità_raffreddamento* determina il valore della “temperatura” *T* in funzione del tempo.



Algoritmi Genetici

Sono algoritmi evolutivi ispirati ai modelli dell'evoluzione delle specie in natura.

Utilizzano il principio della selezione naturale che favorisce gli individui di una popolazione che sono più adatti ad uno specifico ambiente per sopravvivere e riprodursi.

Popolazione iniziale:

- k stati/individui generati casualmente
- ogni individuo è rappresentato come una stringa (come il DNA)

Esempio: 24 bit o posizione nelle colonne ("24748552") stato delle 8 regine

Ogni individuo rappresenta una soluzione con il corrispondente valore della funzione di valutazione denominato **fitness**.

Esempio: numero di coppie di regine che **non** si attaccano



Algoritmi Genetici: operatori

I tre principali **operatori** che caratterizzano questi algoritmi sono:

- **Selezione**: per selezionare gli individui che diventeranno i *genitori* della generazione successiva. I genitori sono solitamente scelti in base al loro valore di fitness tra tutti gli individui o un loro sottoinsieme scelto a caso
- **Mutazione**: Il **tasso di mutazione**, che determina la frequenza con cui la *prole* presenta mutazioni casuali della loro rappresentazione. Una volta generata la prole, ogni bit che la compone viene invertito con probabilità uguale al tasso di mutazione.
- **Ricombinazione**: parti degli stati genitore sono usati per generare uno stato figlio.



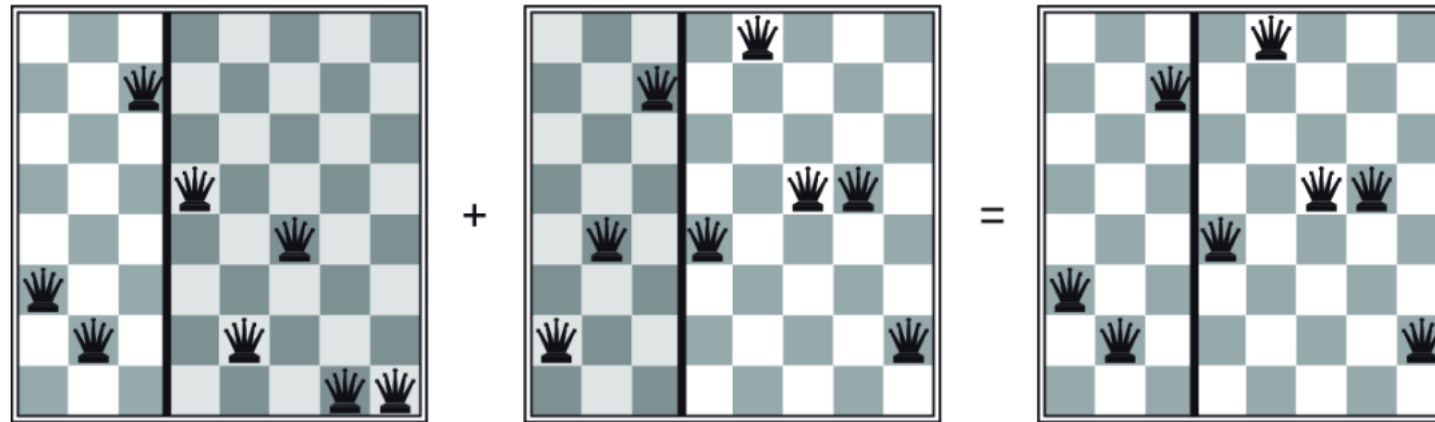
Algoritmi Genetici: esempio delle 8 regine



- La **popolazione iniziale** in (a) è ordinata in base alla funzione di **fitness** in (b), dando come risultato le coppie di (c)
- Per ciascuna coppia viene scelto un punto di **cross-over** e i due genitori producono due figli scambiandosi pezzi (d)
- Viene infine effettuata una **mutazione** casuale che dà luogo alla prossima generazione (e)

La popolazione ottenuta dovrebbe essere migliore. La cosa si ripete fino ad ottenere stati abbastanza buoni (stati obiettivo)

Algoritmi Genetici: esempio delle 8 regine (nascita di un figlio)



- La figura rappresenta i primi due genitori della figura precedente.
- Le parti chiare sono passate al figlio mentre le parti grigie si perdono nel passo di crossover

Solitamente all'inizio del processo la popolazione è molto diversificata quindi il crossover effettua grandi passi nello spazio degli stati all'inizio del processo (come nel simulated annealing).

Dopo molte generazioni di selezione verso un valore della fitness più alto, la popolazione è meno diversificata e si effettuano passi più piccoli

Algoritmi Genetici: la funzione Riproduzione

```
function Riproduzione(genitore1, genitore2) returns un individuo  
    n ← Lunghezza(genitore1)  
    c ← numero casuale fra 1 e n  
    return Concatena(Sottostringa(genitore1, 1, c), Sottostringa(genitore2, c + 1, n))
```

Le stringhe che rappresentano i due genitori sono ricombinate ad ottenere un figlio. Il punto di cross-over **c** viene scelto in modo casuale.



Algoritmi Genetici: Algoritmo

```
function Algoritmo-Genetico(popolazione, fitness) returns un individuo
  repeat
    pesi  $\leftarrow$  Pesato-Da(popolazione, fitness)
    popolazione2  $\leftarrow$  lista vuota
    for i = 1 to Dimensione(popolazione) do
      genitore1, genitore2  $\leftarrow$  Selezione-Casuale-Pesata(popolazione, pesi, 2)
      figlio  $\leftarrow$  Riproduzione(genitore1, genitore2)
      if (piccola probabilità casuale) then figlio  $\leftarrow$  Mutazione(figlio)
      aggiungi figlio a popolazione2
    popolazione  $\leftarrow$  popolazione2
  until esiste in popolazione un individuo con fitness sufficientemente alto, o è passato
  abbastanza tempo
  return l'individuo migliore nella popolazione in base alla fitness
```



Algoritmi Genetici: Osservazioni

Gli algoritmi genetici funzionano meglio quando sono presenti degli schemi che corrispondono a componenti significative di una soluzione.

Esempio: se la stringa rappresenta un'antenna, gli schemi potrebbero rappresentare componenti quali riflettori e deflettori. Un buon componente probabilmente sarà utile in una varietà di progetti diversi.

Il successo degli algoritmi genetici richiede una cura particolare nella progettazione della rappresentazione.

Gli algoritmi genetici hanno un ruolo significativo nell'ampio panorama dei metodi di ottimizzazione, in particolare per problemi strutturati complessi come la configurazione di circuiti e più recentemente per l'evoluzione dell'architettura delle reti neurali deep

